Tugas Personal ke-2

COSC6025 - Data Structures and Algorithm Analysis

Week 7: Linked List, Stack dan Queue (LO 3)

Nama: Risky Saputra Siahaan

NIM: 2902816222

Topik 1: Linked List

A. Soal Review

1. Konsep dasar Linked List dan perbedaannya dengan Array

Linked List adalah struktur data linear yang terdiri dari node-node. Setiap node menyimpan data dan pointer ke node berikutnya, dan node-node tersebut tersebar di memori lalu dihubungkan lewat pointer. Array menyimpan elemen secara berurutan di blok memori yang bersebelahan, berukuran tetap, dan diakses langsung lewat indeks (O(1)). Pada Linked List, untuk mencapai elemen ke-n harus dilakukan penelusuran dari head (O(n)).

2. Kelebihan dan kekurangan Linked List dibanding Array

- Kelebihan: ukuran dinamis (bisa bertambah dan berkurang saat program berjalan), insert dan delete tidak perlu menggeser elemen, tidak perlu mengalokasikan memori besar di awal.
- Kekurangan: tidak bisa akses acak (harus traversal), butuh memori tambahan untuk pointer, pencarian O(n), dan kurang ramah cache karena memori tidak berurutan.

3. Cara kerja operasi insert dan delete

- Insert: buat node baru. Di awal: baru.next = head, lalu head = baru. Di akhir: telusuri sampai node terakhir, lalu terakhir.next = baru. Di tengah: baru.next = after, lalu before.next = baru.
- Delete: cari node sebelum target, lalu before.next = target.next sehingga target terlepas dari rantai. Hapus head: head = head.next. Hapus node terakhir: telusuri sampai node sebelum terakhir, lalu next = null. Memori node dibersihkan oleh garbage collector Java.

B. Soal Case: Pengelolaan Daftar Buku Perpustakaan

Program dibuat dengan Single Linked List yang terdiri dari tiga file: Node.java (data satu buku), LinkedList.java (operasi push, pop, cari, tampil), dan Main.java (menu dan input Scanner). Ringkasan fitur:

- Tambah Buku (Push): input kodeBuku, judul, penulis. Validasi kodeBuku maksimal 5 karakter dan tidak boleh ganda. Data masuk di akhir daftar.
- Hapus Buku (Pop): menghapus buku terakhir. Jika kosong tampil "Tidak ada data untuk dihapus."
- Cari Buku: berdasarkan kodeBuku. Jika tidak ada tampil "Buku tidak ditemukan."
- Tampilkan Semua Buku: sesuai urutan input, lengkap dengan total buku di akhir.
- Keluar: jika data kurang dari 5 buku, program memberi peringatan minimal 5 buku dan meminta konfirmasi.

Source Code

Node.java

```
// Node untuk menyimpan satu data buku
public class Node {
    String kodeBuku;
    String judul;
    String penulis;
    Node next; // penunjuk ke node berikutnya

    public Node(String kodeBuku, String judul, String penulis) {
```

```
        this.kodeBuku = kodeBuku;
        this.judul = judul;
        this.penulis = penulis;
        this.next = null;
    }
}
```

LinkedList.java

```
// Single Linked List untuk daftar buku perpustakaan
public class LinkedList {
    private Node head;
    private int size;

    public LinkedList() {
        head = null;
        size = 0;
    }

    public int getSize() {
        return size;
    }

    public boolean isEmpty() {
        return head == null;
    }

    // Cek apakah kodeBuku sudah dipakai (mencegah data ganda)
    public boolean sudahAda(String kode) {
        return cari(kode) != null;
    }

    // PUSH: tambah data baru di AKHIR daftar
    public void push(String kode, String judul, String penulis) {
        Node baru = new Node(kode, judul, penulis);
        if (head == null) {
            head = baru; // daftar kosong, node baru jadi head
        } else {
            Node bantu = head;
            while (bantu.next != null) { // telusuri sampai node terakhir
                bantu = bantu.next;
            }
            bantu.next = baru; // sambungkan di belakang node terakhir
        }
        size++;
    }

    // POP: hapus data buku TERAKHIR. Mengembalikan false jika daftar kosong
    public boolean pop() {
        if (head == null) {
            return false;
        }
        if (head.next == null) { // hanya satu node
            head = null;
        } else {
            Node bantu = head;
            while (bantu.next.next != null) { // berhenti di node sebelum terakhir
                bantu = bantu.next;
            }
            bantu.next = null; // putus node terakhir
        }
        size--;
        return true;
    }

    // CARI: temukan buku berdasarkan kodeBuku (tidak peduli huruf besar/kecil)
    public Node cari(String kode) {
        Node bantu = head;
        while (bantu != null) {
            if (bantu.kodeBuku.equalsIgnoreCase(kode)) {
                return bantu;
            }
            bantu = bantu.next;
        }
        return null;
    }

    // TAMPIL: cetak semua buku sesuai urutan input + total di akhir
    public void tampil() {
        if (head == null) {
```

```
            System.out.println("Daftar buku masih kosong.");
            System.out.println("Total Buku: 0");
            return;
        }
        System.out.println("Daftar Buku:");
        Node bantu = head;
        while (bantu != null) {
            System.out.println("Kode: " + bantu.kodeBuku
                    + " | Judul: " + bantu.judul
                    + " | Penulis: " + bantu.penulis);
            bantu = bantu.next;
        }
        System.out.println("Total Buku: " + size);
    }
}
```

Main.java

```
import java.util.Scanner;

// Program pengelolaan daftar buku perpustakaan (Single Linked List)
public class Main {
    static final int MIN_BUKU = 5;      // jumlah data minimal
    static final int MAX_KODE = 5;      // panjang maksimal kodeBuku

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        LinkedList daftar = new LinkedList();
        boolean jalan = true;

        while (jalan) {
            System.out.println();
            System.out.println("===== SISTEM DATA BUKU =====");
            System.out.println("1. Tambah Buku");
            System.out.println("2. Hapus Buku");
            System.out.println("3. Cari Buku");
            System.out.println("4. Lihat Semua Buku");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            String pilih = sc.nextLine().trim();

            switch (pilih) {
                case "1":
                    tambahBuku(sc, daftar);
                    break;
                case "2":
                    // Hapus buku terakhir
                    if (daftar.isEmpty()) {
                        System.out.println("Tidak ada data untuk dihapus.");
                    } else {
                        daftar.pop();
                        System.out.println("Buku terakhir berhasil dihapus!");
                    }
                    break;
                case "3":
                    cariBuku(sc, daftar);
                    break;
                case "4":
                    daftar.tampil();
                    break;
                case "5":
                    // Validasi: data minimal 5 buku sebelum keluar
                    if (daftar.getSize() < MIN_BUKU) {
                        System.out.println("Data baru " + daftar.getSize()
                                + " buku. Minimal harus " + MIN_BUKU + " buku.");
                        System.out.print("Tetap keluar? (y/n): ");
                        String jawab = sc.nextLine().trim();
                        if (jawab.equalsIgnoreCase("y")) {
                            jalan = false;
                        }
                    } else {
                        jalan = false;
                    }
                    break;
                default:
                    System.out.println("Menu tidak valid, pilih 1-5.");
            }
        }
        System.out.println("Terima kasih, program selesai.");
        sc.close();
```

```
    }

    // Input data buku + validasi
    static void tambahBuku(Scanner sc, LinkedList daftar) {
        System.out.print("Masukkan Kode Buku: ");
        String kode = sc.nextLine().trim();
        if (kode.isEmpty() || kode.length() > MAX_KODE) {
            System.out.println("Gagal: kode buku wajib diisi dan maksimal "
                    + MAX_KODE + " karakter.");
            return;
        }
        if (daftar.sudahAda(kode)) {
            System.out.println("Gagal: kode buku " + kode + " sudah terdaftar.");
            return;
        }
        System.out.print("Masukkan Judul: ");
        String judul = sc.nextLine().trim();
        System.out.print("Masukkan Penulis: ");
        String penulis = sc.nextLine().trim();
        if (judul.isEmpty() || penulis.isEmpty()) {
            System.out.println("Gagal: judul dan penulis tidak boleh kosong.");
            return;
        }
        daftar.push(kode, judul, penulis);
        System.out.println("Data berhasil ditambahkan!");
    }

    // Cari buku berdasarkan kode
    static void cariBuku(Scanner sc, LinkedList daftar) {
        System.out.print("Masukkan Kode Buku: ");
        String kode = sc.nextLine().trim();
        Node hasil = daftar.cari(kode);
        if (hasil == null) {
            System.out.println("Buku tidak ditemukan.");
        } else {
            System.out.println("Buku ditemukan!");
            System.out.println("Kode: " + hasil.kodeBuku
                    + " | Judul: " + hasil.judul
                    + " | Penulis: " + hasil.penulis);
        }
    }
}
```

Topik 2: Stack dan Queue

A. Soal Review

1. Perbedaan Stack dan Queue

Stack memakai prinsip LIFO (Last In First Out): elemen yang terakhir masuk keluar lebih dulu, dan akses hanya lewat satu ujung (top). Queue memakai prinsip FIFO (First In First Out): elemen yang pertama masuk keluar lebih dulu, masuk lewat belakang (rear) dan keluar lewat depan (front).

2. Operasi utama Stack dan Queue

- Stack: push (tambah ke top), pop (ambil dan hapus top), peek (lihat top), isEmpty, size.
- Queue: enqueue (tambah ke rear), dequeue (ambil dan hapus front), peek/front (lihat elemen depan), isEmpty, size.

3. Contoh penerapan nyata

- Stack: undo/redo pada editor teks, tombol back pada browser, call stack pemanggilan fungsi, pengecekan kurung seimbang.
- Queue: antrian kasir atau bank, antrian print job, penjadwalan proses pada sistem operasi, message queue.

B. Soal Case: Sistem Kasir Toko

Program memakai dua class terpisah, Queue untuk antrian pelanggan (FIFO) dan Stack untuk riwayat transaksi (LIFO), serta class Node untuk menyimpan data (kode, nama, total). Ringkasan fitur:

- Tambah Antrian (enqueue): input nomorAntrian, namaPelanggan, totalBelanja. Validasi jumlah antrian maksimal 5, nomor tidak boleh ganda, total harus angka lebih dari 0.
- Layani Pelanggan: dequeue pelanggan terdepan, lalu datanya di-push ke stack riwayat.
- Tampilkan Antrian: dari depan ke belakang beserta jumlahnya.
- Lihat Riwayat Transaksi: dari terbaru ke terlama (LIFO), beserta total transaksi dan pendapatan.
- Keluar dari program.

Source Code

Node.java

```
// Node untuk menyimpan data pelanggan / transaksi
public class Node {
    String kode;      // nomor antrian
    String nama;      // nama pelanggan
    double total;     // total belanja
    Node next;

    public Node(String kode, String nama, double total) {
        this.kode = kode;
        this.nama = nama;
        this.total = total;
        this.next = null;
    }
}
```

Queue.java

```
// Queue (FIFO) berbasis linked list untuk antrian pelanggan
public class Queue {
    private Node front; // depan antrian (dilayani duluan)
    private Node rear;  // belakang antrian (tempat pelanggan baru masuk)
    private int size;

    public Queue() {
        front = rear = null;
        size = 0;
    }

    public int getSize() { return size; }

    public boolean isEmpty() { return front == null; }

    // Cek apakah nomor antrian sudah dipakai
    public boolean sudahAda(String kode) {
        Node bantu = front;
        while (bantu != null) {
            if (bantu.kode.equalsIgnoreCase(kode)) return true;
            bantu = bantu.next;
        }
        return false;
    }

    // ENQUEUE: tambah pelanggan di belakang antrian
    public void enqueue(String kode, String nama, double total) {
        Node baru = new Node(kode, nama, total);
        if (rear == null) {
            front = rear = baru;
        } else {
            rear.next = baru;
            rear = baru;
        }
        size++;
    }

    // DEQUEUE: ambil pelanggan paling depan. Null jika kosong
    public Node dequeue() {
        if (front == null) return null;
        Node keluar = front;
        front = front.next;
        if (front == null) rear = null; // antrian jadi kosong
        keluar.next = null;
```

```
        size--;
        return keluar;
    }

    // Tampilkan seluruh antrian dari depan ke belakang
    public void tampil() {
        if (front == null) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Antrian saat ini:");
        Node bantu = front;
        int no = 1;
        while (bantu != null) {
            System.out.printf("%d. %s | %s | Rp%,.0f%n", no++, bantu.kode, bantu.nama,
bantu.total);
            bantu = bantu.next;
        }
        System.out.println("Jumlah dalam antrian: " + size);
    }
}
```

Stack.java

```
// Stack (LIFO) berbasis linked list untuk riwayat transaksi
public class Stack {
    private Node top; // transaksi terbaru
    private int size;

    public Stack() {
        top = null;
        size = 0;
    }

    public int getSize() { return size; }

    public boolean isEmpty() { return top == null; }

    // PUSH: simpan transaksi baru di paling atas
    public void push(String kode, String nama, double total) {
        Node baru = new Node(kode, nama, total);
        baru.next = top;
        top = baru;
        size++;
    }

    // Tampilkan riwayat dari terbaru (atas) ke terlama (bawah)
    public void display() {
        if (top == null) {
            System.out.println("Belum ada riwayat transaksi.");
            return;
        }
        System.out.println("Riwayat Transaksi (terbaru ke lama):");
        Node bantu = top;
        int no = 1;
        double jumlah = 0;
        while (bantu != null) {
            System.out.printf("%d. %s | %s | Rp%,.0f%n", no++, bantu.kode, bantu.nama,
bantu.total);
            jumlah += bantu.total;
            bantu = bantu.next;
        }
        System.out.println("Total transaksi: " + size);
        System.out.printf("Total pendapatan: Rp%,.0f%n", jumlah);
    }
}
```

Main.java

```
import java.util.Scanner;

// Sistem kasir toko: Queue untuk antrian, Stack untuk riwayat transaksi
public class Main {
    static final int MAX_ANTRIAN = 5; // validasi jumlah data antrian <= 5

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Queue antrian = new Queue();
        Stack riwayat = new Stack();
        boolean jalan = true;
```

```
        while (jalan) {
            System.out.println();
            System.out.println("=== SISTEM KASIR TOKO ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Layani Pelanggan");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Lihat Riwayat Transaksi");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            String pilih = sc.nextLine().trim();

            switch (pilih) {
                case "1":
                    tambahAntrian(sc, antrian);
                    break;
                case "2":
                    layani(antrian, riwayat);
                    break;
                case "3":
                    antrian.tampil();
                    break;
                case "4":
                    riwayat.display();
                    break;
                case "5":
                    jalan = false;
                    break;
                default:
                    System.out.println("Menu tidak valid, pilih 1-5.");
            }
        }
        System.out.println("Terima kasih, program selesai.");
        sc.close();
    }

    // Enqueue pelanggan baru dengan validasi
    static void tambahAntrian(Scanner sc, Queue antrian) {
        if (antrian.getSize() >= MAX_ANTRIAN) {
            System.out.println("Antrian penuh (maksimal " + MAX_ANTRIAN
                    + " pelanggan). Layani pelanggan terlebih dahulu.");
            return;
        }
        System.out.print("Masukkan Nomor Antrian: ");
        String kode = sc.nextLine().trim();
        if (kode.isEmpty()) {
            System.out.println("Gagal: nomor antrian tidak boleh kosong.");
            return;
        }
        if (antrian.sudahAda(kode)) {
            System.out.println("Gagal: nomor antrian " + kode + " sudah ada.");
            return;
        }
        System.out.print("Masukkan Nama Pelanggan: ");
        String nama = sc.nextLine().trim();
        if (nama.isEmpty()) {
            System.out.println("Gagal: nama tidak boleh kosong.");
            return;
        }
        System.out.print("Masukkan Total Belanja: ");
        double total;
        try {
            total = Double.parseDouble(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Gagal: total belanja harus berupa angka.");
            return;
        }
        if (total <= 0) {
            System.out.println("Gagal: total belanja harus lebih dari 0.");
            return;
        }
        antrian.enqueue(kode, nama, total);
        System.out.println("Data pelanggan ditambahkan ke antrian!");
    }

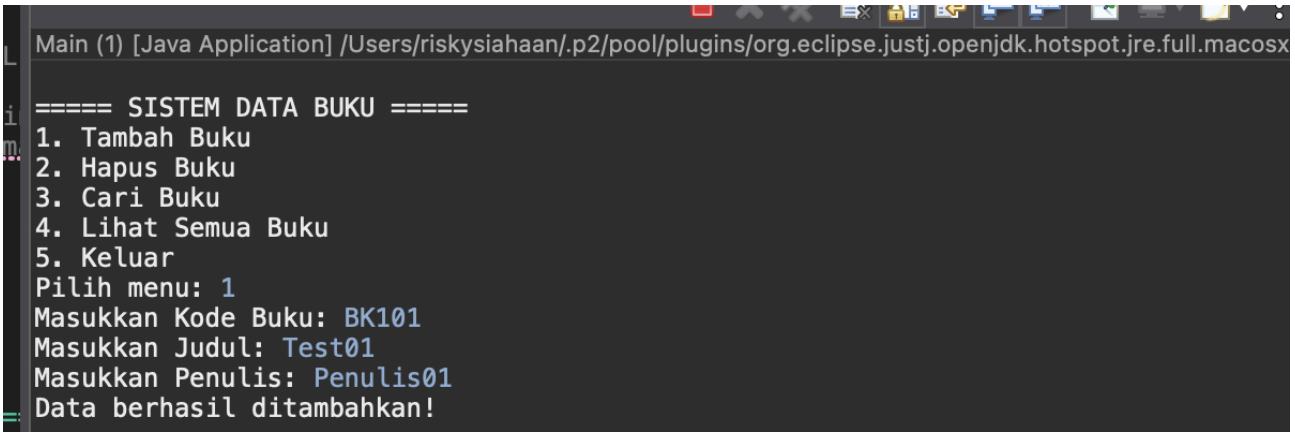
    // Dequeue pelanggan terdepan lalu push ke stack riwayat
    static void layani(Queue antrian, Stack riwayat) {
        Node pelanggan = antrian.dequeue();
        if (pelanggan == null) {
            System.out.println("Tidak ada pelanggan dalam antrian.");
            return;
```

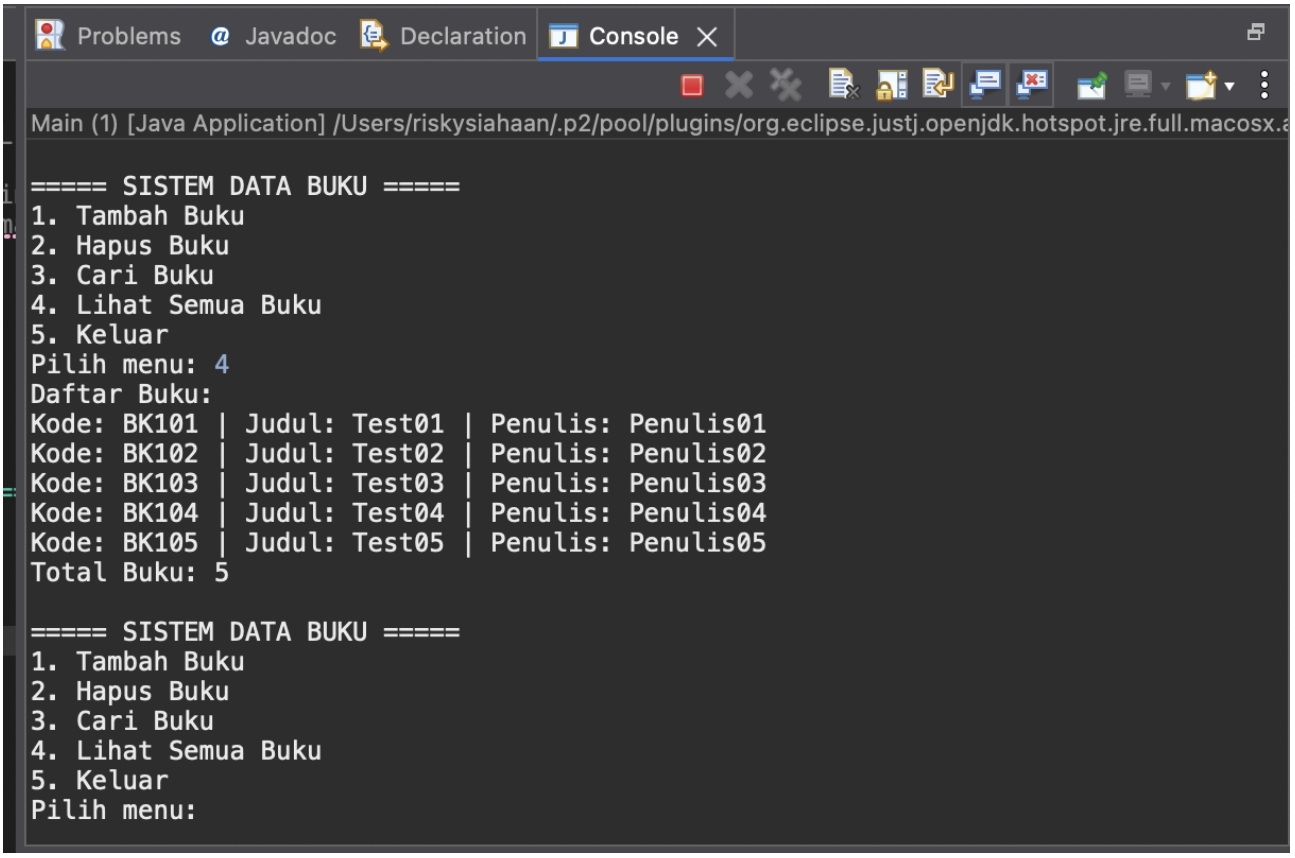
```
        }
        System.out.println("Melayani pelanggan " + pelanggan.kode + " (" + pelanggan.nama + ")");
        riwayat.push(pelanggan.kode, pelanggan.nama, pelanggan.total);
        System.out.println("Transaksi disimpan ke riwayat.");
    }
}
```

Bukti Eksekusi Program

Screenshot hasil eksekusi

Topik 1





```
===== SISTEM DATA BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 3
Masukkan Kode Buku: BK106
Buku tidak ditemukan.
```

```
===== SISTEM DATA BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 2
Buku terakhir berhasil dihapus!
```

Screenshot hasil eksekusi

Topik 2

```
5. Keluar
Pilih menu: 1
Masukkan Nomor Antrian: A005
Masukkan Nama Pelanggan: Duong
Masukkan Total Belanja: 556500
Data pelanggan ditambahkan ke antrian!
```

```
=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 3
Antrian saat ini:
1. A001 | Pham | Rp95,000
2. A002 | Nguyen | Rp223,000
3. A003 | Trần | Rp334,500
4. A004 | Tran | Rp334,588
5. A005 | Duong | Rp556,500
Jumlah dalam antrian: 5
```

```
=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 2
Melayani pelanggan A001 (Pham)
Transaksi disimpan ke riwayat.
```

```
5. Keluar
Pilih menu: 3
Antrian saat ini:
1. A002 | Nguyen | Rp223,000
2. A003 | Trần | Rp334,500
3. A004 | Tran | Rp334,588
4. A005 | Duong | Rp556,500
Jumlah dalam antrian: 4
```

```
Pilih menu: 4
Riwayat Transaksi (terbaru ke lama):
1. A001 | Pham | Rp95,000
Total transaksi: 1
Total pendapatan: Rp95,000
```

Link Kode Program (Github)

https://github.com/riskyshn/COSC6025036-Personal-Assignment-2